

CallForge — User Manual

A targeted-capture germline analysis pipeline (FASTQ → annotated joint callset → burden)

Bright Adu Genomics & Bioinformatics Core
Noguchi Memorial Institute for Medical Research
University of Ghana, Legon, Ghana

2026-06-14

Contents

1. Introduction	3
2. Concepts	4
3. Pipeline architecture	5
4. Installation	8
4.1 Local install (personal computer)	8
4.2 Shared HPC install (Ubuntu 24.04 SLURM cluster)	8
5. Inputs	10
5.1 Sample sheet (CSV) — <code>--input</code>	10
5.2 CaptureForge handoff — <code>--target_bed</code> (+ metadata)	10
5.3 Reference — <code>--genome_fasta</code>	10
6. Annotation databases	11
6.1 The databases	11
6.2 The genomic-scope gate (fail-loud)	11
6.3 Contig-naming / chr-reconciliation	12
6.4 Fetching databases — <code>bin/fetch_annotation_dbs.sh</code>	12
6.5 Resource auto-discovery	12
6.6 Bring-your-own / non-human databases	12
7. Configuration	13
8. Running — step-by-step scenarios	15
8.1 Quickstart (test profile)	15
8.2 Full human cohort (primary scenario)	15
8.3 Callset-only (no phenotype)	15
8.4 Burden / association run (with phenotype)	15
8.5 A different panel	16
8.6 A different organism	16
8.7 HPC run	16
8.8 Choosing engine alternatives	16
8.9 GIAB validation run	17
9. Quality control	18
9.1 Read & alignment QC (Stages 1–4)	18
9.2 Per-sample QC gate & quarantine (Stage 5)	18
9.3 Cohort QC dashboard & MultiQC (Stage 15)	23
10. Stage details & their plots	24
10.1 SNV/indel calling & filtering (Stages 6–7)	24
10.2 CNV, STR, paralog (Stages 8–10)	24
10.3 Annotation (Stage 11)	29

10.4 Cohort QC (Stage 12)	29
10.5 GIAB validation (Stage 13)	29
10.6 Burden (Stage 14)	29
11. Outputs explained	35
12. Interpreting results	36
13. Real-run checklist	37
14. Troubleshooting	38
15. Extending CallForge	39
16. Reproducibility & provenance	40
17. Glossary & FAQ	41
18. Citation & the CaptureForge ↔ CallForge pairing	42

1. Introduction

CallForge is a disease- and organism-agnostic **Nextflow DSL2** pipeline that turns demultiplexed paired-end FASTQs from a hybridization-capture panel into a high-quality, joint-genotyped, **annotated** callset spanning **SNV/indel**, **CNV**, **STR** and **paralog-aware** variants, and carries through to a **rare-variant burden / association** layer.

CallForge is the **analysis counterpart to CaptureForge** (the probe-design pipeline) and **consumes CaptureForge’s handoff**: the as-built target `final_covered_targets.bed` plus per-gene metadata (design class, CNV callability, paralog flags, STR loci, burden groups). That design-time knowledge drives and labels the matching analysis steps — the **CaptureForge → CallForge interlock**.

Its immediate use is a 59-gene human malaria-susceptibility panel on African-ancestry dried-blood-spot (DBS) samples, but it generalizes to **any panel, cohort, and (where feasible) organism** by changing inputs and configuration only — no code edits.

Who it is for. Bioinformaticians and genomics-core staff running targeted germline analysis who need a reproducible, QC-gated workflow with honest reporting. The pipeline emphasizes **QC at every stage with captioned plots, fail-loud invariants** (reference and annotation-database scope), and a **flag-and-quarantine** QC gate that never silently drops data.

Status note used throughout this manual. CallForge is implemented and verified end-to-end on a small synthetic **test** profile. Where a number is a property of that synthetic data (not of a real cohort), the text and figure captions say so explicitly — *machinery-proven, not data quality*. See §11 (Interpreting results) and §12 (Real-run checklist).

2. Concepts

Targeted-capture germline analysis. A hybridization-capture panel enriches sequencing for a defined set of target regions (here, CaptureForge’s `final_covered_targets.bed`). Reads are aligned to a reference, duplicates marked, and variants called *jointly* across the cohort so that genotypes — including homozygous-reference calls — are consistent and comparable across samples.

Joint genotyping. Per-sample gVCFs (GATK HaplotypeCaller) are combined (GenomicsDBImport or CombineGVCFs) and genotyped together (GenotypeGVCFs). Joint genotyping yields a square cohort matrix and well-calibrated genotypes, which matter for downstream cohort QC and burden testing.

The four variant classes (why each needs its own caller).

Class	Caller	Why a dedicated method
SNV / indel	GATK HaplotypeCaller (or DeepVariant)	local re-assembly of haplotypes
CNV	CNVkit (or GATK gCNV)	read-depth ratios vs a panel-of-normals, not per-base genotypes
STR	ExpansionHunter	repeat-aware genotyping from spanning/flanking reads
paralog-aware	<code>paralog_flag.py</code> confidence layer	multi-mapping ambiguity is a <i>confidence</i> problem, not a new caller

The CaptureForge → CallForge interlock. CaptureForge records, per gene, what it could and could not design well. CallForge consumes that:

- **CNV callability** — depth-callable genes (e.g. GYPC, HP) vs breakpoint-blind genes (e.g. CR1, CFH) label every CNV call’s confidence.
- **STR catalog** — STR-class targets seed the ExpansionHunter variant catalog.
- **Paralog flags** — paralog-ambiguous genes (e.g. CD209, CASP1, CR1, HP, CFH) drive the paralog confidence layer.
- **Burden groups** — collapse units for gene-set burden testing.

The reference invariant. CallForge aligns to the **same no-alt GRCh38 primary assembly CaptureForge designed against**, and asserts it fail-loud: no alt/decoy contigs, and the target BED’s build must match the reference (see §9, §13).

Rare-variant burden testing. Individually rare variants are aggregated per gene (and per gene-set) and tested for association with phenotype, adjusting for covariates and ancestry. CallForge filters to **rare + functional** variants, collapses by **gene** and **CaptureForge burden group**, and runs a gene-burden test (§7, §10).

3. Pipeline architecture

CallForge runs **16 stages (0–15)**. Each stage writes metrics (JSON/TSV), emits captioned plots (PNG+SVG), and feeds MultiQC plus a self-contained cohort QC dashboard. The diagram below is the authoritative map; external inputs (dashed = drives/labels downstream stages) feed in from the side.

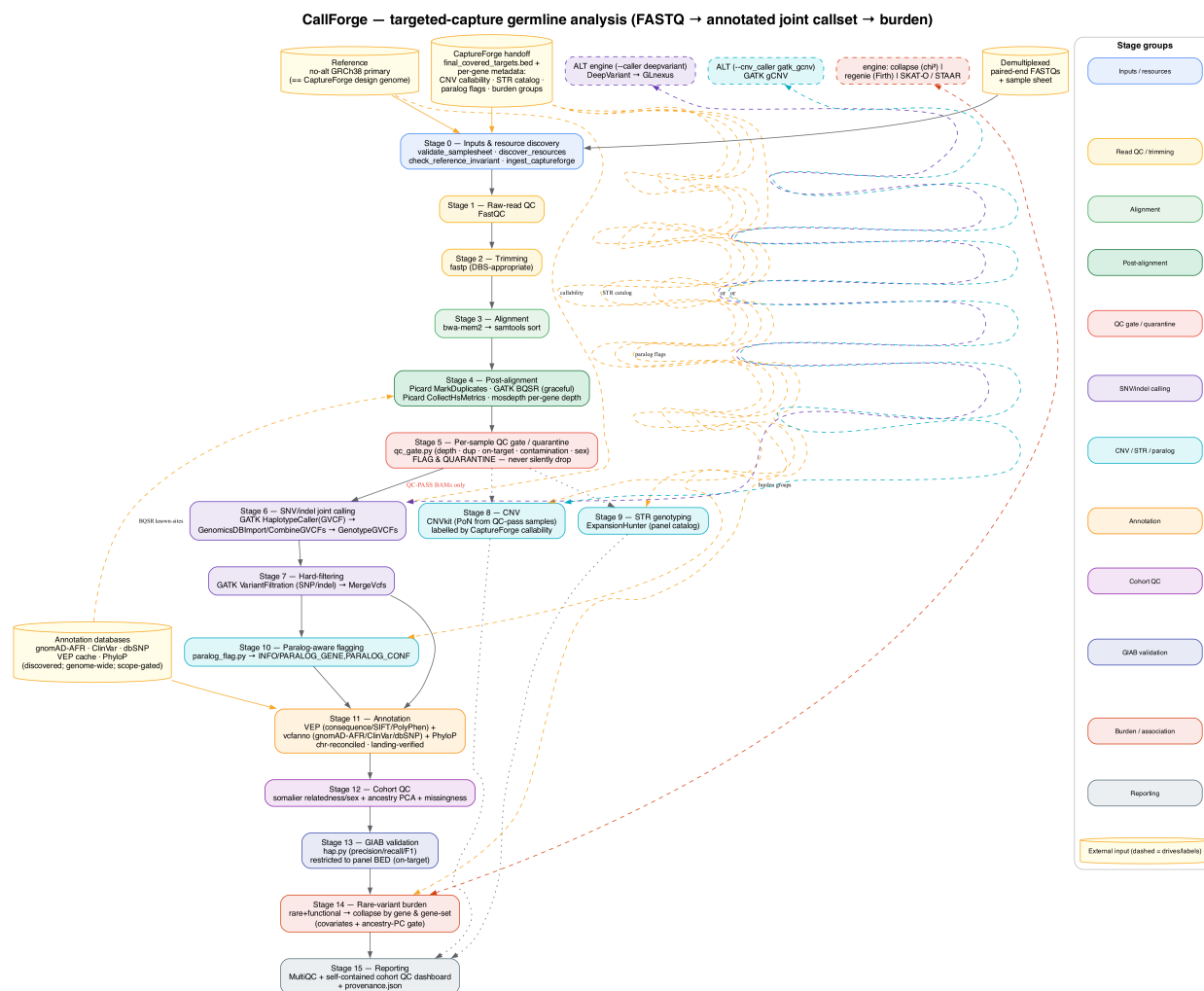


Figure 1: CallForge pipeline DAG — stages colour-coded by group; external inputs (reference, CaptureForge handoff, annotation databases, FASTQs) feed in; alternative engines (DeepVariant+GLnexus, gatk_gcnv, collapse/regenie/SKAT-O) shown as dashed branches.

Stage walkthrough (real inputs → outputs).

#	Stage	Key tool(s)	Main output(s) under results/
0	Inputs & resource discovery	<code>validate_samplesheet</code> , <code>stage0_inputs/discover_resources</code> , <code>check_reference_invariant</code> , <code>gene_metadata.tsv</code> , <code>ingest_captureforge</code>	(manifest, invariant, <code>gene_metadata.tsv</code>)
1	Raw-read QC	FastQC	<code>stage1_rawqc/</code>
2	Trimming	fastp	<code>stage2_trim/</code>
3	Alignment	bwa-mem2 → samtools sort	<code>stage3_align/</code>
4	Post-alignment	Picard MarkDuplicates, GATK BQSR, CollectHsMetrics, mosdepth	<code>stage4_postalign/</code>
5	Per-sample QC gate	<code>qc_gate.py</code>	<code>stage5_qc_gate/</code> (<code>qc_pass.txt</code> , <code>qc_quarantine.txt</code>)
6	SNV/indel joint calling	GATK HaplotypeCaller → GenomicsDBImport → GenotypeGVCFs	<code>stage6_calling/joint.vcf.gz</code>
7	Hard-filtering	GATK VariantFiltration → MergeVcfs	<code>stage7_filter/joint.filtered.vcf</code>
8	CNV	CNVkit	<code>stage8_cnv/</code> (<code>cnv_calls.tsv</code> , callability)
9	STR	ExpansionHunter	<code>stage9_str/</code>
10	Paralog-aware	<code>paralog_flag.py</code>	<code>stage10_paralog/paralog.annotated.vcf</code>
11	Annotation	VEP + vcfanno + PhyloP	<code>stage11_annotation/annotated.vcf</code> <code>variants.flat.tsv</code>
12	Cohort QC	somalier	<code>stage12_cohortqc/</code>
13	GIAB validation	hap.py	<code>stage13_giab/</code>
14	Burden / association	collapse / regenie / SKAT-O	<code>stage14_burden/burden_results.tsv</code>
15	Reporting	MultiQC + dashboard + provenance	<code>stage15_report/</code>

Nextflow / profile model. `main.nf` calls one subworkflow (`subworkflows/callforge.nf`) that wires modules in `modules/`. Behaviour is set by **profiles** composed on the command line: an execution profile (`test`, `mac_local`, `hpc_slurm`) plus an environment engine (`conda`, `docker`, `apptainer`). Each process declares its own pinned conda env (`env/*.yaml`) or a pinned container (VEP, hap.py), so `-profile ... ,conda` or `,apptainer` builds exactly what each stage needs.

Engine options (see §7 for full defaults): SNV/indel `--caller gatk|deepvariant`; joint method `--joint_method genomicsdb|combinegvcfs`; CNV `--cnv_caller cnvkit|gatk_gcnv`;

```
CNVkit segmentation --cnv_segment_method cbs|haar|hmm; burden --burden_engine  
regenie|skat|collapse.
```


4. Installation

CallForge needs **Nextflow** (≥ 23.10) and either conda/mamba or a container engine. Each stage runs in its own pinned environment (`env/*.yaml`) or pinned container, so you do not install tools by hand — `-profile conda` (or `,apptainer`) builds each environment on first use.

4.1 Local install (personal computer)

Common steps.

```
# 1. Nextflow (Java 17+ required)
curl -s https://get.nextflow.io | bash && sudo mv nextflow /usr/local/bin/
# or: mamba create -n nextflow -c bioconda 'nextflow>=23.10'
```

```
# 2. clone CallForge (it lives beside CaptureForge)
cd ~/Documents/Claude/Projects/callforge
```

```
# 3. core environment (others build on demand)
mamba env create -f env/callforge.yaml
```

macOS (Apple Silicon). Some bioconda tools lack `osx-arm64` builds. Two options:

```
# (a) Rosetta-emulated x86 env (documented path; the report flags emulated
↪ stages)
CONDA_SUBDIR=osx-64 mamba env create -f env/callforge.yaml
```

```
# (b) native arm64 - works for most tools, but pin mosdepth >= 0.3.11
#      (mosdepth 0.3.8 has no arm64 build; 0.3.8 + Rosetta is the alternative)
```

VEP and hap.py are run from their **official containers** (robust across platforms); ensure Docker is available and use `-profile <exec>,docker`.

Linux. Native; `mamba env create -f env/*.yaml` resolves cleanly, or use the container route.

Container route. Build the core image once and run containerized:

```
docker build -t callforge:0.1.0 -f env/Dockerfile .
# Apptainer (HPC): apptainer build callforge.sif env/callforge.def
```

4.2 Shared HPC install (Ubuntu 24.04 SLURM cluster)

The production cohort run is intended for the cluster (native linux-64; no Rosetta). The `hpc_slurm` profile uses the SLURM executor (global, cluster-wide across the nodes) and Apptainer for the containerized tools.

```
# shared conda at /hpc/opt/conda
source /hpc/opt/conda/etc/profile.d/conda.sh
mamba env create -p /hpc/opt/conda/envs/callforge -f env/callforge.yaml
mamba env create -p /hpc/opt/conda/envs/callforge-annotate -f env/annotate.yaml
# ... and cohortqc / cnv / str / happy / burden / deepvariant as needed
```

```

# Apptainer image for the core tools (Docker is usually forbidden on clusters)
apptainer build callforge.sif env/callforge.def
export APPTAINER_BINDPATH=/data/refs,/data/annotation_db,$HOME/.vep    # mount
    ↪ resources

```

```

# run
nextflow run main.nf -profile hpc_slurm,apptainer -params-file params.full.yaml \
    --slurm_partition compute \
    --slurm_account mylab \
    --scratch_dir /scratch/$USER

```

An **Lmod** module + launcher wrapper is the recommended front door (set PATH to the shared Nextflow, source the shared conda, set APPTAINER_BINDPATH); see docs/hpc_deployment.md. conf/hpc_slurm.config sets a shared conda cache so each env builds once cluster-wide; --slurm_partition / --slurm_account / --scratch_dir are overridable. Heavy steps (index, align, large labels) get bigger slots.

5. Inputs

5.1 Sample sheet (CSV) — `--input`

Columns: `sample_id`, `fastq_1`, `fastq_2`, `sex`, `phenotype`, `covariate_*`, `batch`. Example:

```
sample_id,fastq_1,fastq_2,sex,phenotype,covariate_age,batch
S001,/data/fq/S001_R1.fastq.gz,/data/fq/S001_R2.fastq.gz,F,case,41,b1
S002,/data/fq/S002_R1.fastq.gz,/data/fq/S002_R2.fastq.gz,M,control,55,b1
HG002,/data/fq/HG002_R1.fastq.gz,/data/fq/HG002_R2.fastq.gz,M,control,30,b2
```

- `sample_id` unique; `fastq_1`/`fastq_2` are the paired-end reads (validated to exist and differ). `sex` is M/F/U (free tokens normalized).
- **phenotype is required only for the burden layer** (case/control or quantitative). Without it, the pipeline runs through the annotated callset and **skips burden with a message** (`burden_meta.json` records `status: skipped_no_phenotype`).
- `covariate_*` columns (any number) become burden covariates; `batch` is carried for QC.

Validation (`validate_samplesheet.py`) writes `samplesheet.valid.csv` and `samplesheet_summary.json` (with `burden_eligible`).

5.2 CaptureForge handoff — `--target_bed` (+ metadata)

Point CallForge at CaptureForge’s as-built outputs:

```
target_bed:
  ↪ /path/captureforge/results/full_run/stage9_qc/final_covered_targets.bed
captureforge_dir: /path/captureforge/results/full_run # auto-finds metrics.json
  ↪ + baits.csv
paralog_genes: "CD209,CASP1,CR1,HP,CFH"
```

`final_covered_targets.bed` uses `GENE|class` names (e.g. `ATP2B4|coding`, `GYPC|cnv`). `metrics.json` supplies CNV callability; `baits.csv` an off-target signal. If CaptureForge outputs are absent, supply `--gene_metadata` (a per-gene TSV), `--paralog_genes`, and `--str_catalog` directly. `ingest_captureforge.py` produces `gene_metadata.tsv` with columns `gene`, `classes`, `...`, `is_cnv_target`, `cnv_callable`, `cnv_callable_reason`, `is_str_target`, `str_loci`, `is_paralog`, `off_target_frac`, `low_coverage`, `burden_group`.

5.3 Reference — `--genome_fasta`

The **no-alt GRCh38 primary assembly** the panel was designed against (Ensembl-named, e.g. `1,2,...,22,X,Y,MT`). `--genome_build GRCh38`. CallForge builds `.fai/.dict/bwa-mem2` index if absent and asserts the reference invariant (§9).

6. Annotation databases

Annotation is the most resource-sensitive stage and is treated carefully: CallForge **discovers** local resources, **validates their genomic scope** (fail-loud on region-limited databases), and **reconciles contig naming** so values are never silently missed. Nothing is downloaded unless a resource is missing **and** required **and** you opt in (`--allow_download`).

6.1 The databases

Database	Provides	Stage(s)	Param	Scope requirement
gnomAD (AFR)	population allele frequencies, incl. African-ancestry AF_afr	11 (vcfanno), 14 (rare filter)	<code>--gnomad_vcf</code> , <code>--gnomad_af_fields</code> AF_afr	genome-wide; scope-gated
ClinVar	clinical significance (CLNSIG, CLNDN, CLNREVSTAT, ...)	11	<code>--clinvar_vcf</code>	genome-wide; scope-gated
dbSNP	rsIDs (ID column)	11 (rsID), 4 (BQSR known-site)	<code>--dbsnp_vcf</code> , <code>--known_sites</code>	genome-wide; scope-gated
VEP cache	consequence, transcript, SIFT/PolyPhen, canonical/MANE	11	<code>--vep_cache</code> , <code>--vep_release</code>	species + assembly match
PhyloP	conservation score (bigWig)	11	<code>--phylop_bw</code>	genome-wide

Namespacing. gnomAD INFO fields are written back as `gnomad_*` (e.g. `gnomad_AF_afr`) so the **cohort's own AF/AC/AN are never overwritten** — the classic annotation enrichment bug is designed out.

6.2 The genomic-scope gate (fail-loud)

A region-subset database (e.g. a single-locus gnomAD) silently returns “no AF / no ClinVar” outside its region — which a downstream filter would misread as *rare-novel*, corrupting burden testing. `discover_resources.py` therefore validates, per annotation VCF, that its data **spans the target panel's contigs** (tabix-probed, chr/no-chr reconciled), checks the **BGZF-EOF marker** (rejects truncated downloads), and **fails loud** on a found-but-unfit database:

```
[discover_resources] UNFIT (found but region-limited): gnomad_vcf: data spans  
↪ only
```

1/19 target contigs (frac=0.0526 < 0.9); region-subset DB - replace with
↪ genome-wide.

[discover_resources] FAIL: ... Refusing to under-annotate.

Tune with `--scope_min` (default 0.9); override only with `--ignore_unfit_resources` (not advised) or `--allow_download`.

6.3 Contig-naming / chr-reconciliation

The design reference / callset is **Ensembl-numeric** (1), gnomAD and PhyloP are usually **chr-prefixed** (chr1), and NCBI ClinVar is Ensembl-numeric. CallForge reconciles naming **per resource**: the query is renamed to a resource's style only where it differs, then renamed back — and the annotation step then **verifies the values actually landed** (`verify_annotation.py`, `fail-loud`), so a contig-name mismatch can never silently drop an annotation.

6.4 Fetching databases — `bin/fetch_annotation_dbs.sh`

A resumable downloader for genome-wide gnomAD v4.1 + ClinVar (GRCh38):

```
# panel-slice mode (recommended for a targeted panel; a few hundred MB)
MODE=panel \
PANEL_BED=/path/final_covered_targets.bed \
OUTDIR=/path/annotation_db \
  bash bin/fetch_annotation_dbs.sh
```

```
# full genome-wide mode (very large; use on HPC with a fast network)
MODE=full DATASETS="genomes" bash bin/fetch_annotation_dbs.sh
```

Panel mode merges nearby intervals to cut HTTP round-trips, slices each chromosome with per-chromosome retry, and is resumable (re-run to continue, not restart). **dbSNP is not fetched by this script** — supply a genome-wide dbSNP separately (it is also a BQSR known-site).

6.5 Resource auto-discovery

`discover_resources.py` scans `--resource_dirs` (default `$HOME,/usr/local/share,/opt`), validates each resource (build, index, scope, integrity), and writes `stage0_inputs/resource_manifest.json`. Missing human-only resources are reported and the dependent step skips gracefully. Explicit `--vep_cache/--gnomad_vcf/--clinvar_vcf/...` overrides take precedence and are still validated.

6.6 Bring-your-own / non-human databases

CallForge is organism-generic: VEP runs by `--species` (point at that species' VEP cache), and `vcfanno` reads a **bring-your-own-databases TOML** (`--vcfanno_toml`). Human-only resources (gnomAD/ClinVar/BQSR known-sites) **skip gracefully when absent**. To register a new organism: set `--species`, `--genome_fasta` (that build), `--vep_cache`, and a `--vcfanno_toml` listing whatever frequency/clinical databases exist; leave the human DB params unset.

7. Configuration

Set parameters via `-params-file params.yaml` (see `params.example.yaml`) or `--key value`. Profiles set executor + resource ceilings: **test** (3–4 synthetic samples, minutes), **mac_local** (full, one machine), **hpc_slurm** (SLURM + Apptainer/shared-conda).

Key tunables (real defaults).

Param	Default	Meaning
<code>--caller</code>	<code>gatk</code>	SNV/indel engine: <code>gatk</code> or <code>deepvariant</code>
<code>--joint_method</code>	<code>genomicsdb</code>	<code>genomicsdb</code> or <code>combinegvcfs</code>
<code>--ploidy</code>	<code>2</code>	HaplotypeCaller ploidy
<code>--cnv_caller</code>	<code>cnvkit</code>	<code>cnvkit</code> or <code>gatk_gcnv</code>
<code>--cnv_method</code>	<code>hybrid</code>	CNVkit hybrid (capture) or <code>amplicon</code>
<code>--cnv_segment_method</code>	<code>cbs</code>	<code>cbs</code> (R/DNAcopy), <code>haar</code> (pure-python), <code>hmm</code>
<code>--min_mean_target_depth</code>	<code>30</code>	QC gate: min mean target depth ($\times$)
<code>--max_dup_rate</code>	<code>0.40</code>	QC gate: max duplication fraction
<code>--min_on_target</code>	<code>0.40</code>	QC gate: min on-target fraction
<code>--max_contamination</code>	<code>0.03</code>	QC gate: max VerifyBamID2 FREEMIX
<code>--enforce_sex_check</code>	<code>true</code>	quarantine on sex mismatch (when assessable)
<code>--snp_filter_expr</code>	GATK WGS/WES default	SNP hard-filter JEXL
<code>--indel_filter_expr</code>	GATK WGS/WES default	indel hard-filter JEXL
<code>--scope_min</code>	<code>0.9</code>	min fraction of target contigs a DB must span
<code>--known_sites</code>	<code>null</code>	BQSR known-sites (dbSNP + Mills + 1000G)
<code>--somalier_sites</code>	<code>null</code>	somalier sites VCF (1000G for the build)
<code>--str_catalog</code>	<code>null</code>	ExpansionHunter catalog (else built from STR targets)
<code>--burden_engine</code>	<code>regenie</code>	<code>regenie</code> , <code>skat</code> , or <code>collapse</code>
<code>--burden_af_max</code>	<code>0.01</code>	rare threshold on gnomAD-AFR AF
<code>--burden_csq</code>	LoF + missense + splice + inframe set	qualifying consequences
<code>--n_ancestry_pcs</code>	<code>4</code>	ancestry PCs offered as covariates (PC-stability gate applies)

Param	Default	Meaning
<code>--vep_image</code>	ensembl-vep 110	VEP container (tag below)
<code>--happy_image</code>	hap.py 0.3.12	hap.py container (tag below)

The container defaults are `ensemblorg/ensembl-vep:release_110.0` (VEP) and `jmcdani20/hap.py:v0.3.12` (hap.py).

Hard-filter thresholds are the GATK WGS/WES defaults. Revisit them on real targeted data (panel Ti/Tv, pass rates, and QUAL/DP distributions guide tuning).

8. Running — step-by-step scenarios

All multi-flag commands use backslash continuation (one flag per line). On a local machine use `,conda` (and `,docker` for the VEP/hap.py containers); on the cluster use `,apptainer`.

8.1 Quickstart (test profile)

```
bash test/make_test_data.sh
nextflow run main.nf -profile test,docker
```

This builds a tiny self-contained fixture (mini-genome + synthetic reads) and runs all 16 stages in minutes. Use it to confirm your install.

8.2 Full human cohort (primary scenario)

```
# 1. write the sample sheet (§5.1) with phenotype if you want burden

# 2. confirm resources discover + pass the scope gate; fetch genome-wide
#    gnomAD/ClinVar if missing (§6.4); supply a genome-wide dbSNP

# 3. run (authoring machine)
nextflow run main.nf \
  -profile mac_local,conda \
  -params-file params.full.yaml

# 3'. or the cluster
nextflow run main.nf \
  -profile hpc_slurm,apptainer \
  -params-file params.full.yaml
```

`params.full.yaml` wires the reference, the CaptureForge handoff, resource directories, and engine choices; set `input:` to your cohort sample sheet. Inspect `results/stage5_qc_gate/qc_scorecard.tsv`, then the annotated callset and the dashboard.

8.3 Callset-only (no phenotype)

Omit `phenotype` (or set `--run_burden false`). The pipeline runs through Stage 11 (annotated callset) + cohort QC + GIAB and **skips burden**; `burden_meta.json` records the skip. Everything else (annotated VCF, per-variant TSV, CNV/STR/paralog) is produced.

8.4 Burden / association run (with phenotype)

```
nextflow run main.nf \
  -profile hpc_slurm,apptainer \
  -params-file params.full.yaml \
```



```

--run_burden true \
--burden_engine regenie \
--burden_af_max 0.01 \
--n_ancestry_pcs 4 \
--covariates covariate_age,covariate_sex

```

Burden filters to rare+functional variants, collapses by gene and CaptureForge burden group, and tests with covariates + ancestry PCs. The **ancestry-PC stability gate** drops the PCs and runs unadjusted (recorded in provenance) if there are too few PCA sites for the cohort — read `burden_results.tsv` (its `engine` column states the engine) with the QQ / Manhattan plots and the validity caveats (§11).

8.5 A different panel

Swap the CaptureForge handoff — no code change:

```

nextflow run main.nf -profile mac_local,conda \
--input samplesheet.csv \
--genome_fasta /ref/GRCh38_noalt.fa \
--target_bed /panelB/final_covered_targets.bed \
--captureforge_dir /panelB/results/full_run

```

8.6 A different organism

```

nextflow run main.nf -profile mac_local,conda \
--input samplesheet.csv \
--genome_build <BUILD> \
--genome_fasta /ref/<species>_noalt.fa \
--target_bed /panel/final_covered_targets.bed \
--species <vep_species> \
--vep_cache /vep/<species> \
--vcfanno_toml /db/<species>.toml \
--run_burden false

```

Human-only resources (gnomAD/ClinVar/BQSR) skip gracefully. Minimum inputs: sample sheet, no-alt reference for that build, target BED, a VEP cache (or `--vep_mode gtf` with a `--gtf`), and a vcfanno TOML for any frequency/clinical databases you have.

8.7 HPC run

See §4.2. `-profile hpc_slurm,apptainer`; SLURM is global/cluster-wide; resources are copied to a shared path and bound into Apptainer.

8.8 Choosing engine alternatives

```

--caller deepvariant           # DeepVariant + GLnexus (container) instead of
↪ GATK
--cnv_caller gatk_gcncv       # GATK gCNV instead of CNVkit

```

```
--burden_engine skat          # SKAT-O / STAAR (R) instead of regenie
--burden_engine collapse      # chi-square collapsing (small-N /
↪ dependency-free)
```

Validate before production. DeepVariant+GLnexus, regenie, and SKAT-O/STAAR are wired but have **not yet been executed end-to-end** (the synthetic test cohort is too small). Exercise them on a real/realistic cohort before relying on them. The tested engines are GATK/GenomicsDB (calling) and collapse (burden).

8.9 GIAB validation run

Add a GIAB control to the sample sheet and supply truth:

```
nextflow run main.nf -profile mac_local,conda -params-file params.full.yaml \
  --giab_control_id HG002 \
  --giab_truth_vcf /giab/HG002_v4.2.1_benchmark.vcf.gz \
  --giab_truth_bed /giab/HG002_v4.2.1_benchmark.bed
```

hap.py scores precision/recall/F1 **restricted to the panel BED** (on-target only), so off-target truth cannot distort the panel's measured sensitivity.

9. Quality control

CallForge’s QC philosophy: **a plot at every applicable stage** (each with a one-line caption), gates that **flag and quarantine** rather than silently drop, and a single cohort QC dashboard plus MultiQC. The figures below are from the synthetic **test** run — they prove the machinery; real numbers come from a real cohort.

9.1 Read & alignment QC (Stages 1–4)

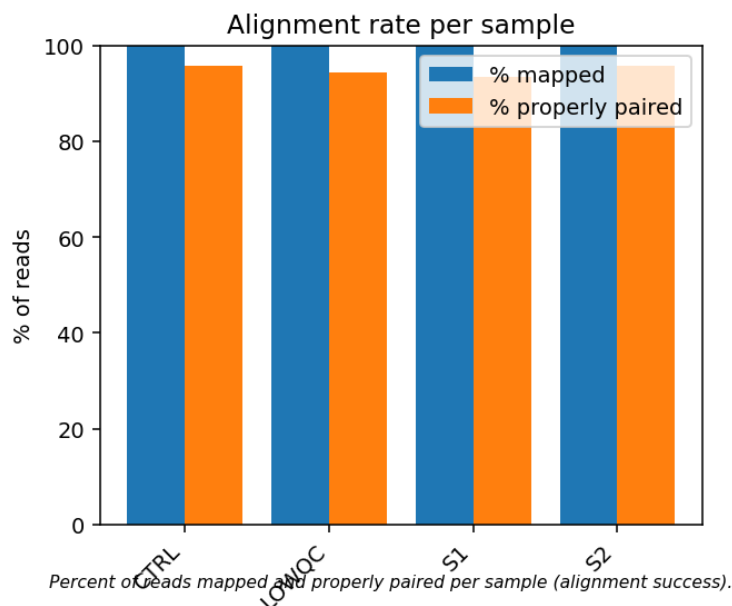


Figure 2: Percent of reads mapped and properly paired per sample (alignment success).

`CollectHsMetrics` (vs the target BED) gives on-target %, fold-enrichment, % target $\geq 10/20/30\times$, and uniformity (`FOLD_80`); `mosdepth` gives per-target and **per-gene depth** in the CaptureForge track style. Duplicates are marked by coordinate (correct for capture). BQSR runs when known-sites are available and **skips gracefully** otherwise.

9.2 Per-sample QC gate & quarantine (Stage 5)

`qc_gate.py` aggregates the per-sample metrics and applies thresholds (depth, dup, on-target, contamination, sex). Failing samples are **flagged and QUARANTINED** — written to `qc_quarantine.txt` with reasons, **excluded from joint calling and from burden counts**, but never silently dropped. Metrics that cannot be assessed (contamination without a `VerifyBamID2` SVD panel; sex on an autosomal panel with no X/Y targets) are recorded **NA**, not failed.

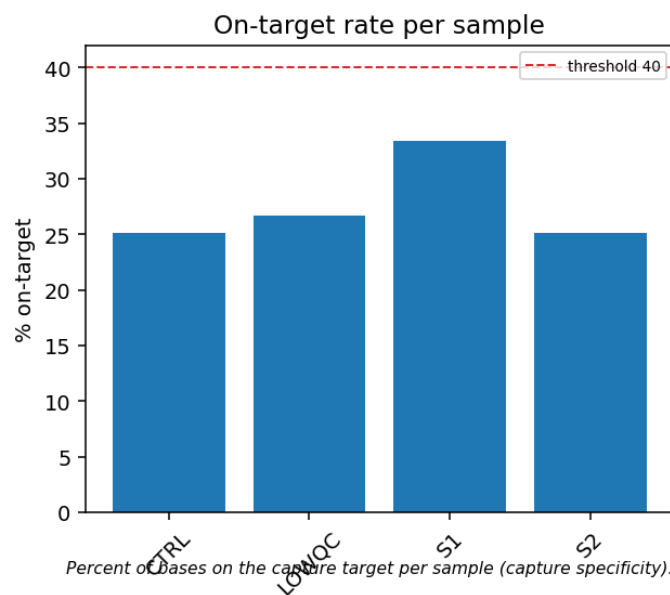


Figure 3: Percent of bases on the capture target per sample (capture specificity).

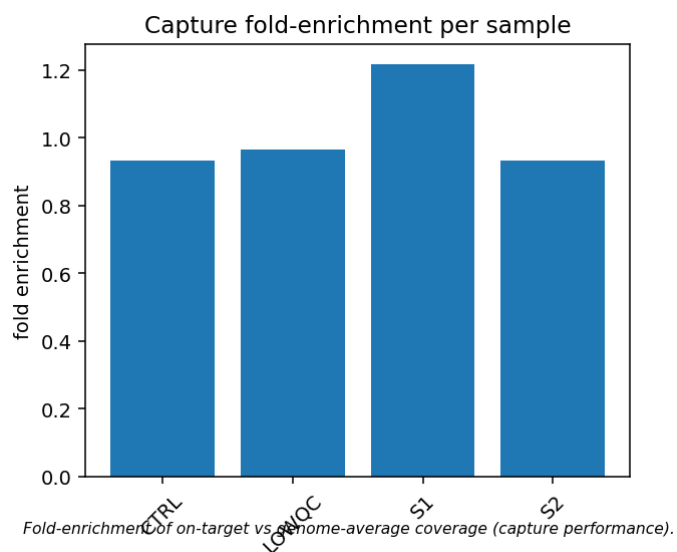


Figure 4: Fold-enrichment of on-target vs genome-average coverage (capture performance).

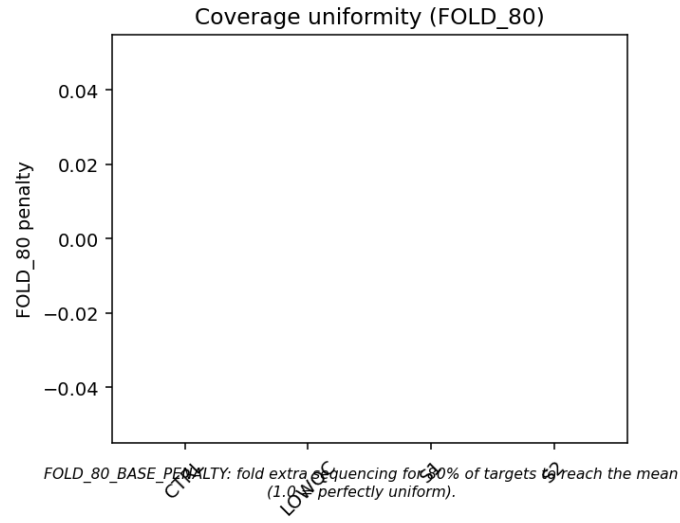


Figure 5: FOLD_80_BASE_PENALTY: fold extra sequencing for 80% of targets to reach the mean (1.0 = perfectly uniform).

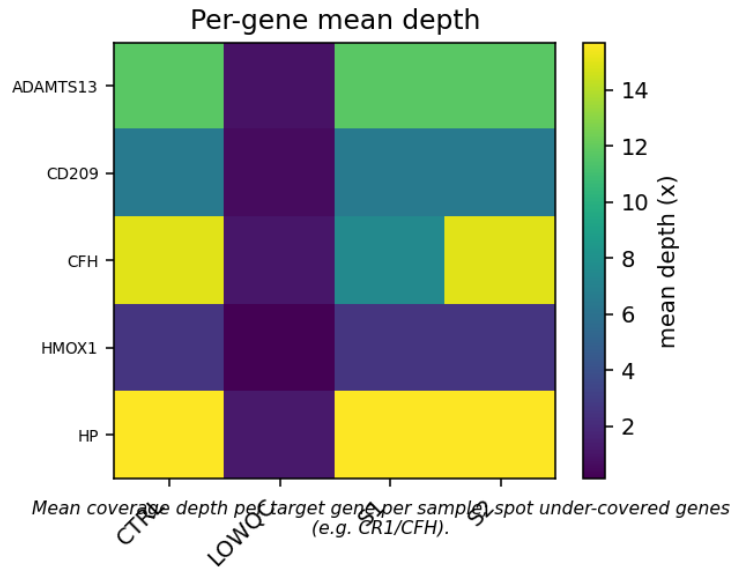


Figure 6: Mean coverage depth per target gene per sample; spot under-covered genes (e.g. CR1/CFH).

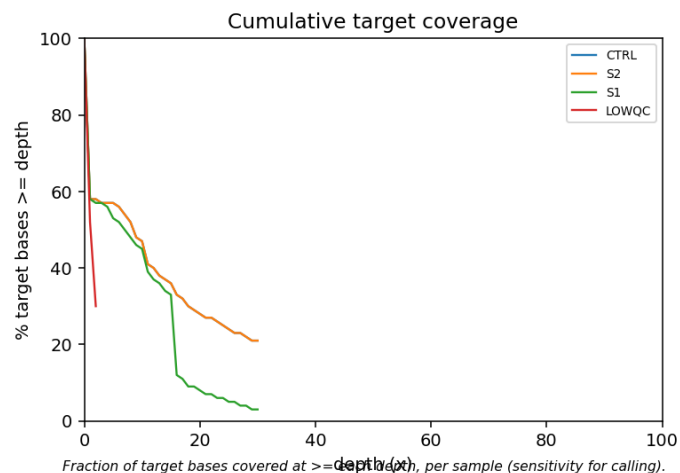


Figure 7: Fraction of target bases covered at $\geq$ each depth, per sample (sensitivity for calling).

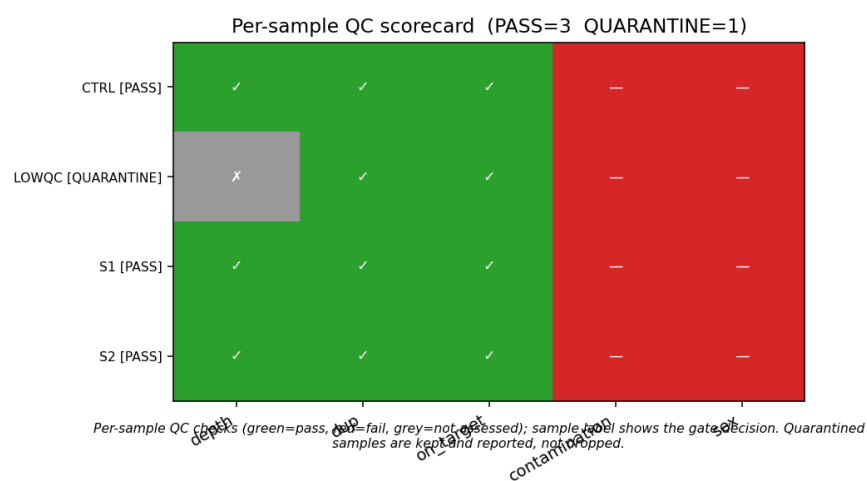


Figure 8: Per-sample QC checks (green=pass, red=fail, grey=not assessed); sample label shows the gate decision. Quarantined samples are kept and reported, not dropped.

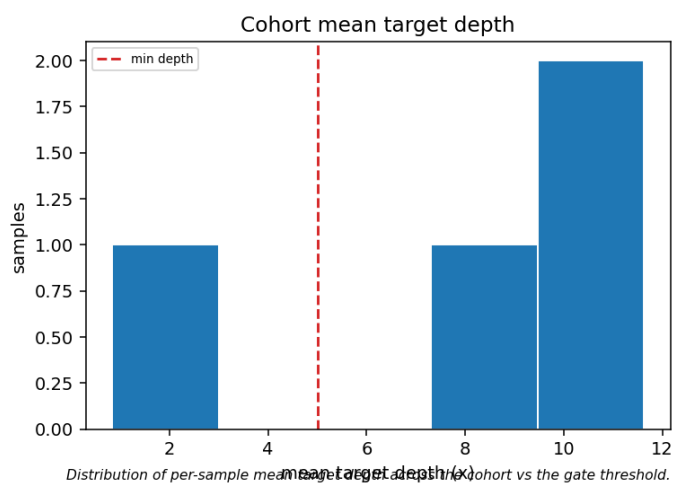


Figure 9: Distribution of per-sample mean target depth across the cohort vs the gate threshold.

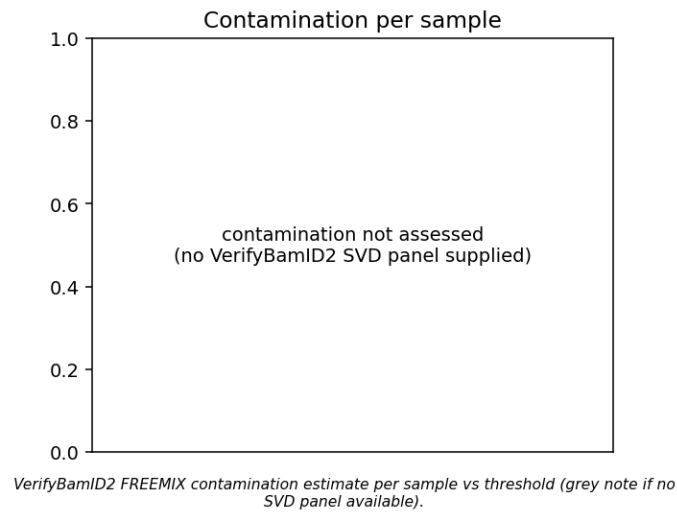


Figure 10: VerifyBamID2 FREEMIX contamination estimate per sample vs threshold (grey note if no SVD panel available).

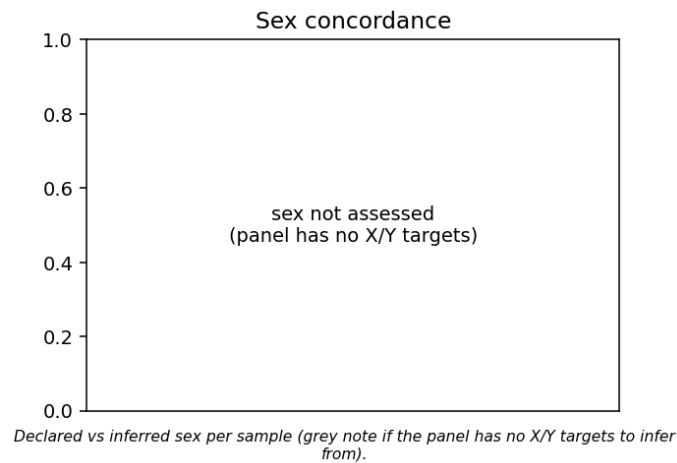


Figure 11: Declared vs inferred sex per sample (grey note if the panel has no X/Y targets to infer from).

9.3 Cohort QC dashboard & MultiQC (Stage 15)

`make_dashboard.py` builds `stage15_report/cohort_qc_dashboard.html` — a **self-contained** HTML embedding every captioned figure (base64) grouped by stage, plus the headline tables (resource manifest, QC scorecard, annotation landing, GIAB, burden). `multiqc_report.html` aggregates the per-tool outputs (FastQC, fastp, Picard, samtools, mosdepth).

10. Stage details & their plots

10.1 SNV/indel calling & filtering (Stages 6–7)

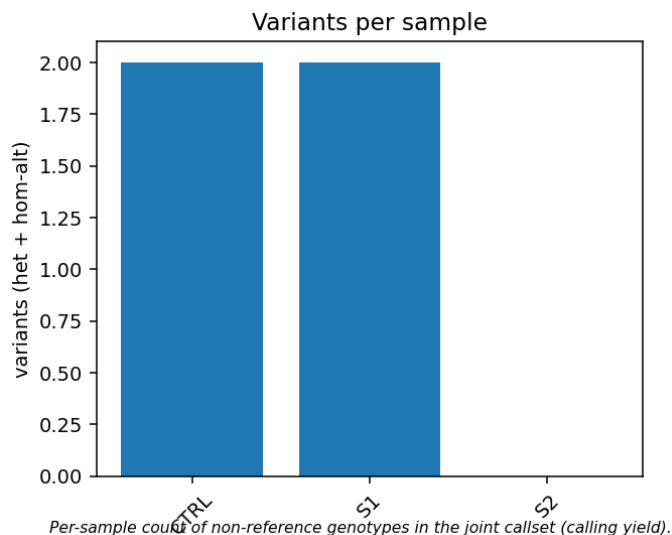


Figure 12: Per-sample count of non-reference genotypes in the joint callset (calling yield).

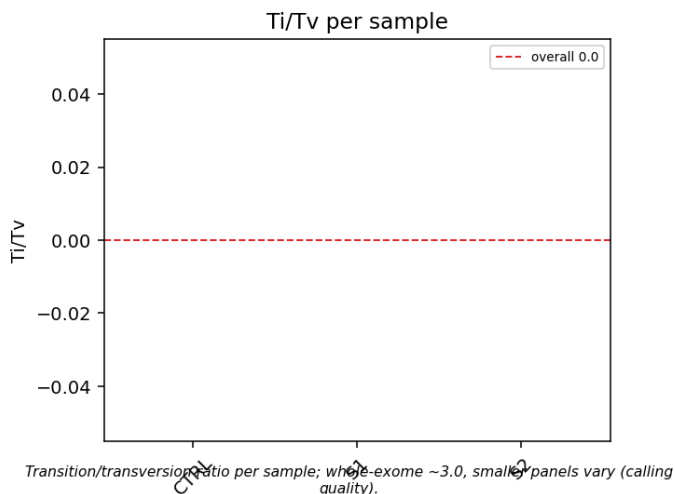


Figure 13: Transition/transversion ratio per sample; whole-exome ~3.0, smaller panels vary (calling quality).

10.2 CNV, STR, paralog (Stages 8–10)

CNV calls (`cnv_calls.tsv`) carry a `cnv_callable` and `confidence` column from CaptureForge metadata — a call at a breakpoint-blind gene (CR1/CFH) is labelled `breakpoint_blind_low_confidence`, not silently trusted.

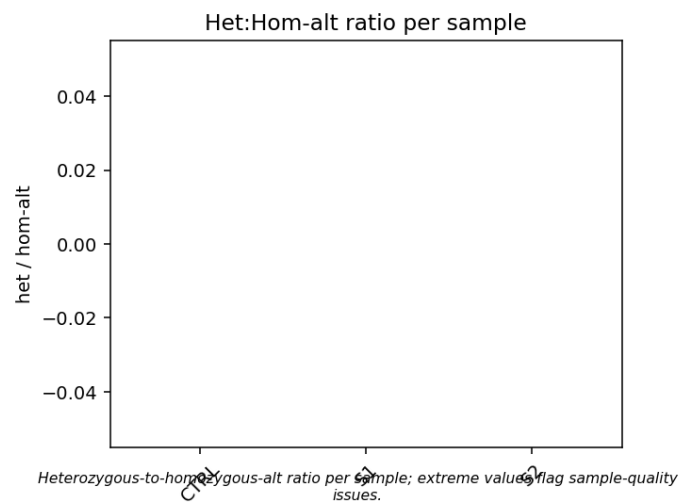


Figure 14: Heterozygous-to-homozygous-alt ratio per sample; extreme values flag sample-quality issues.

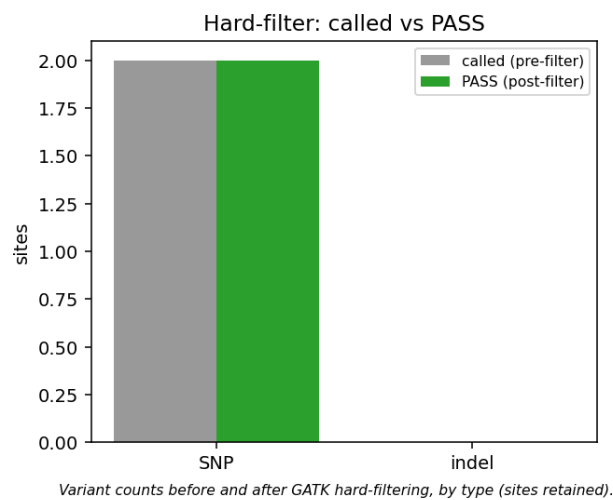


Figure 15: Variant counts before and after GATK hard-filtering, by type (sites retained).

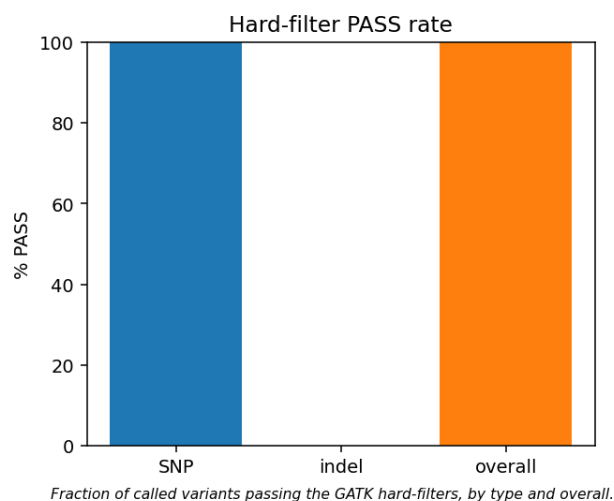


Figure 16: Fraction of called variants passing the GATK hard-filters, by type and overall.

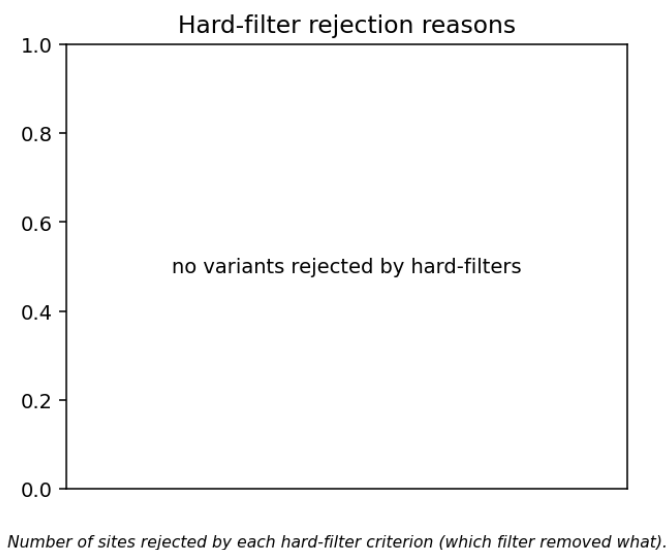


Figure 17: Number of sites rejected by each hard-filter criterion (which filter removed what).

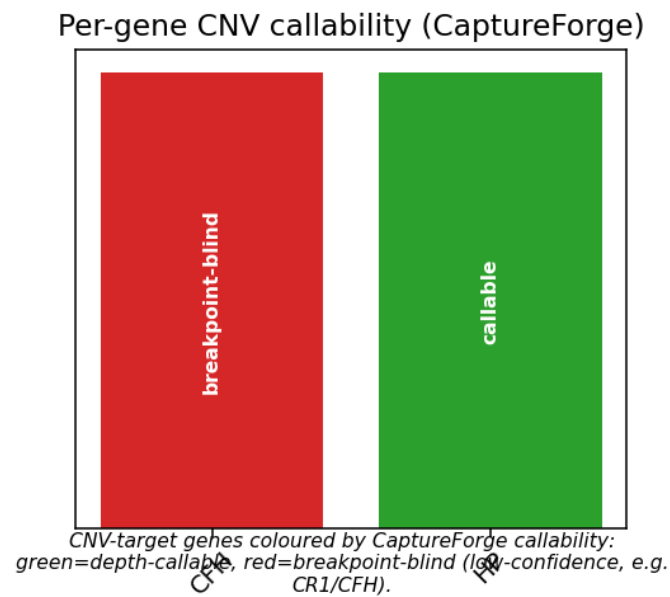


Figure 18: CNV-target genes coloured by CaptureForge callability: green=depth-callable, red=breakpoint-blind (low-confidence, e.g. CR1/CFH).

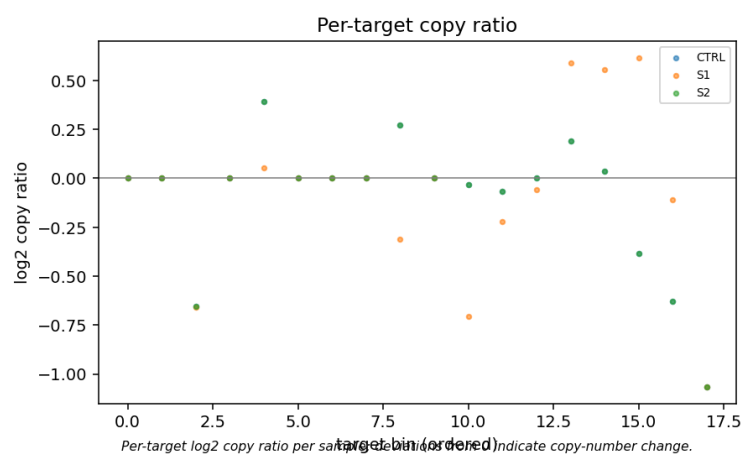


Figure 19: Per-target log2 copy ratio per sample; deviations from 0 indicate copy-number change.

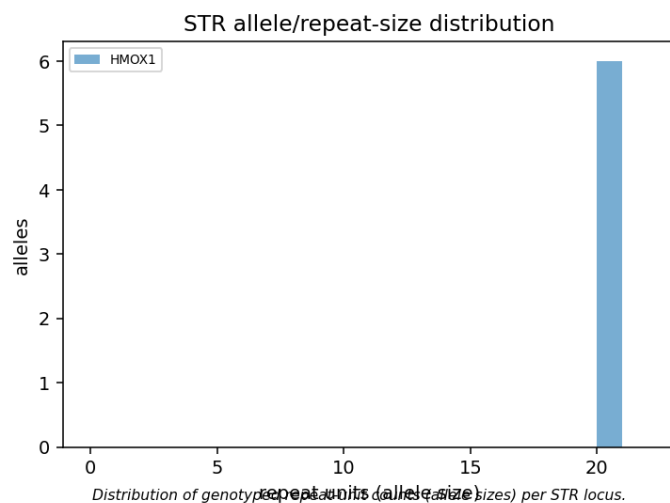


Figure 20: Distribution of genotyped repeat-unit counts (allele sizes) per STR locus.

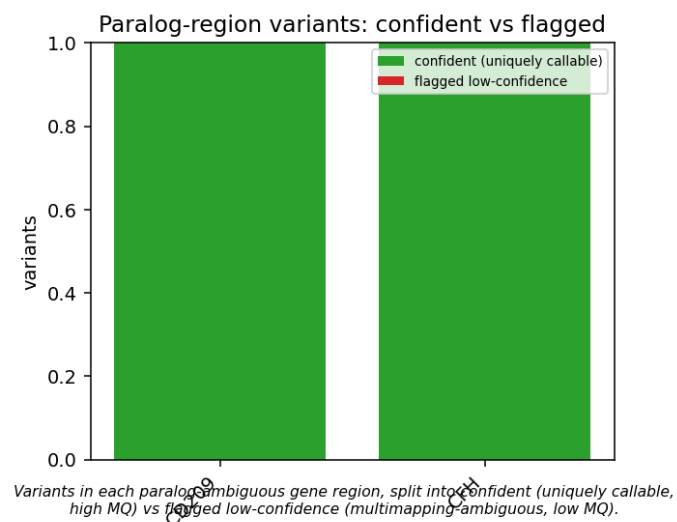


Figure 21: Variants in each paralog-ambiguous gene region, split into confident (uniquely callable, high MQ) vs flagged low-confidence (multimapping-ambiguous, low MQ).

Paralog-aware flagging writes INFO/PARALOG_GENE and INFO/PARALOG_CONF into `paralog.annotated.vcf.gz` so each variant in a paralog region carries its confidence.

10.3 Annotation (Stage 11)

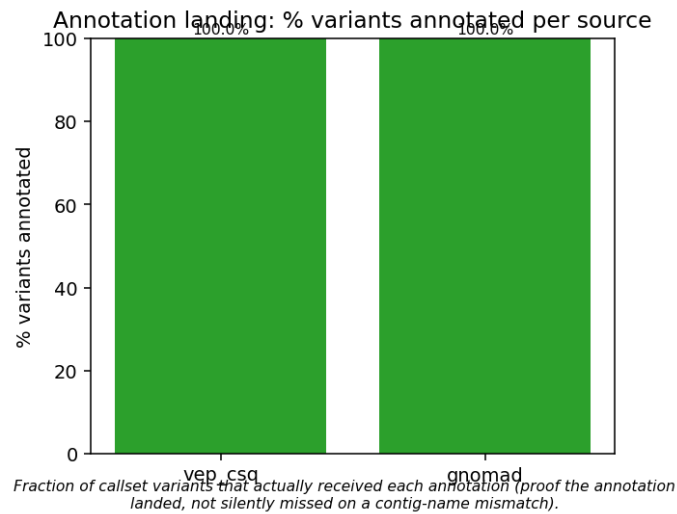


Figure 22: Fraction of callset variants that actually received each annotation (proof the annotation landed, not silently missed on a contig-name mismatch).

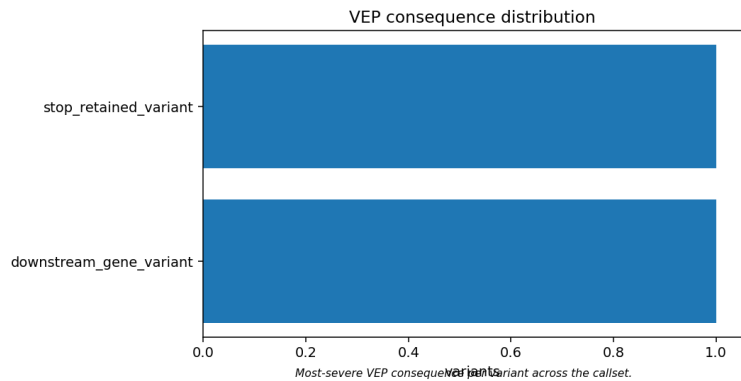


Figure 23: Most-severe VEP consequence per variant across the callset.

10.4 Cohort QC (Stage 12)

10.5 GIAB validation (Stage 13)

10.6 Burden (Stage 14)

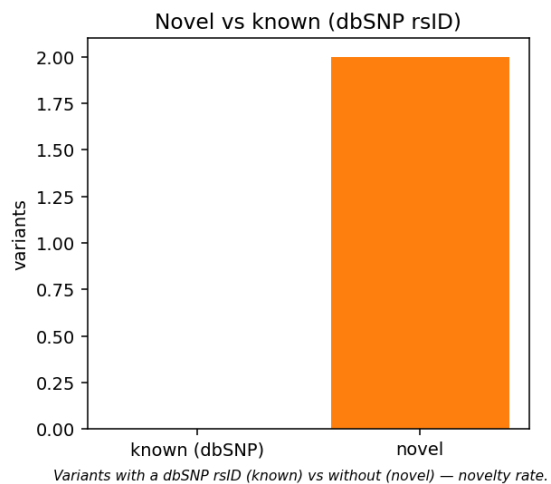


Figure 24: Variants with a dbSNP rsID (known) vs without (novel) — novelty rate.

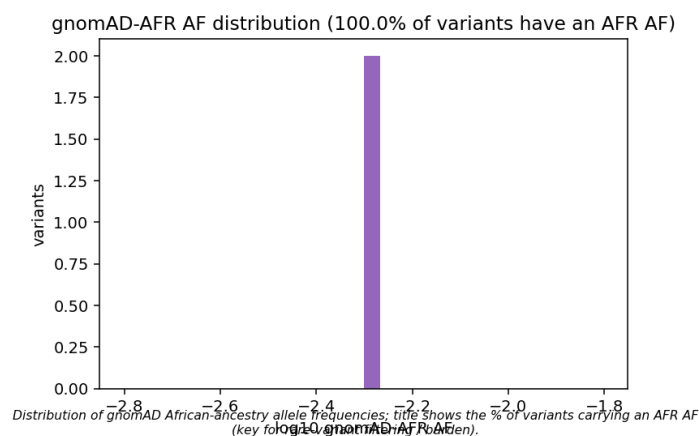


Figure 25: Distribution of gnomAD African-ancestry allele frequencies; title shows the % of variants carrying an AFR AF (key for rare-variant filtering / burden).

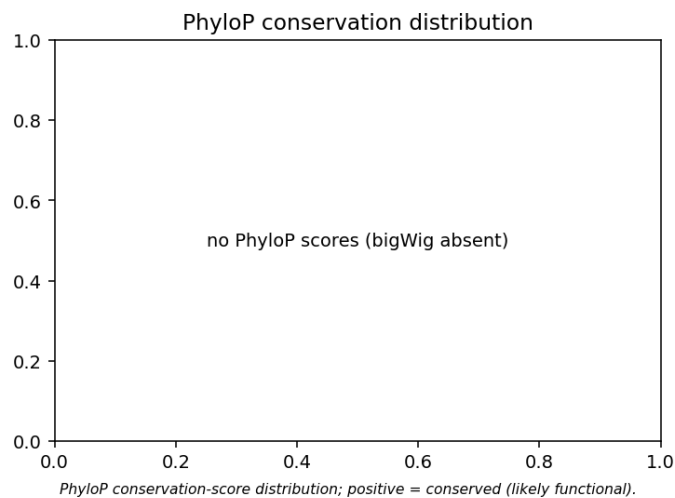


Figure 26: PhyloP conservation-score distribution; positive = conserved (likely functional).

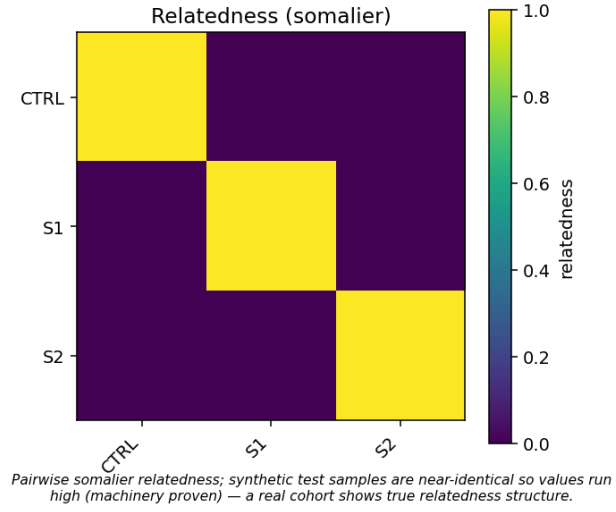


Figure 27: Pairwise somalier relatedness; synthetic test samples are near-identical so values run high (machinery proven) — a real cohort shows true relatedness structure.

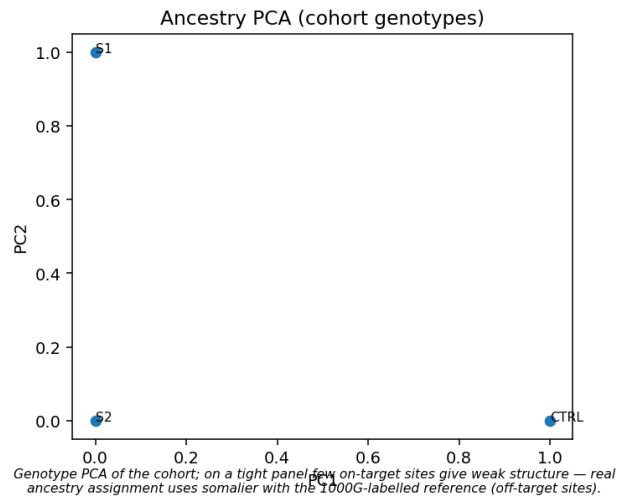


Figure 28: Genotype PCA of the cohort; on a tight panel few on-target sites give weak structure — real ancestry assignment uses somalier with the 1000G-labelled reference (off-target sites).

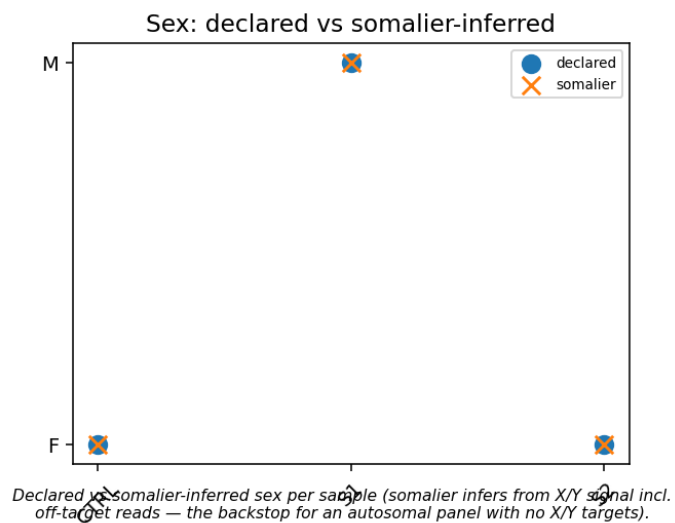


Figure 29: Declared vs somalier-inferred sex per sample (somalier infers from X/Y signal incl. off-target reads — the backstop for an autosomal panel with no X/Y targets).

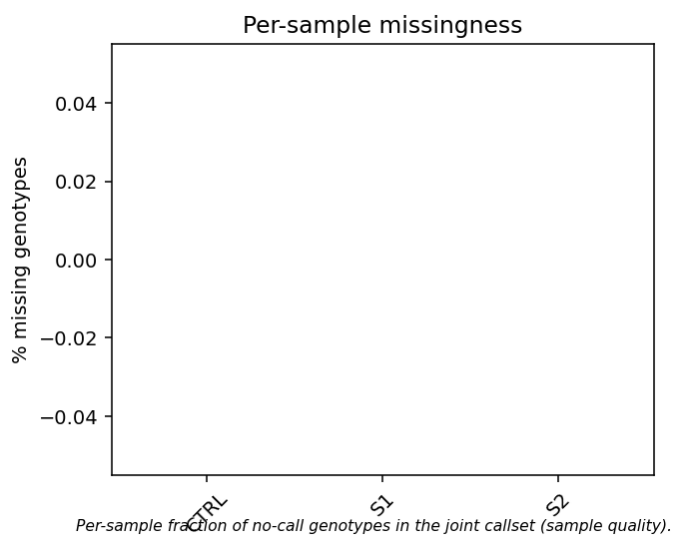


Figure 30: Per-sample fraction of no-call genotypes in the joint callset (sample quality).

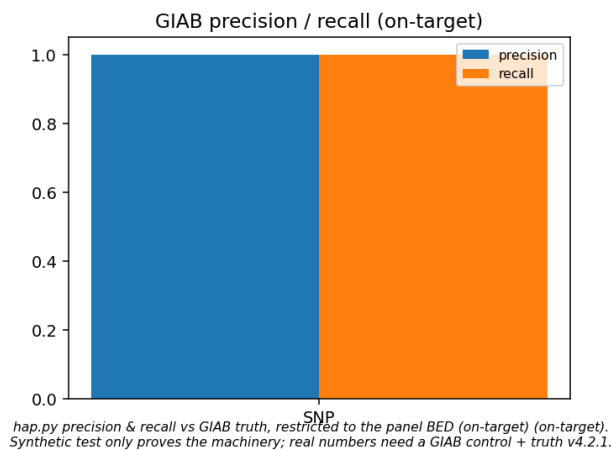


Figure 31: hap.py precision & recall vs GIAB truth, restricted to the panel BED (on-target). Synthetic test only proves the machinery; real numbers need a GIAB control + truth v4.2.1.

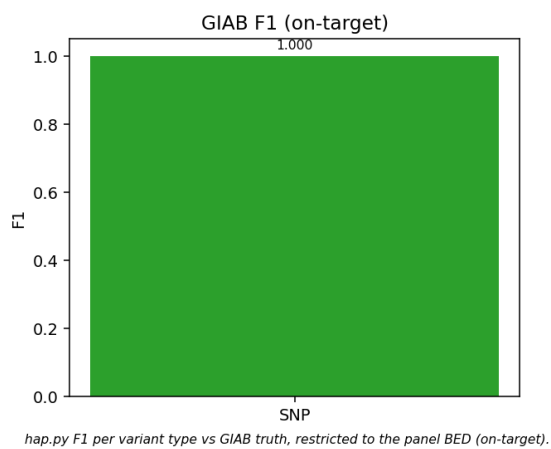


Figure 32: hap.py F1 per variant type vs GIAB truth, restricted to the panel BED (on-target).

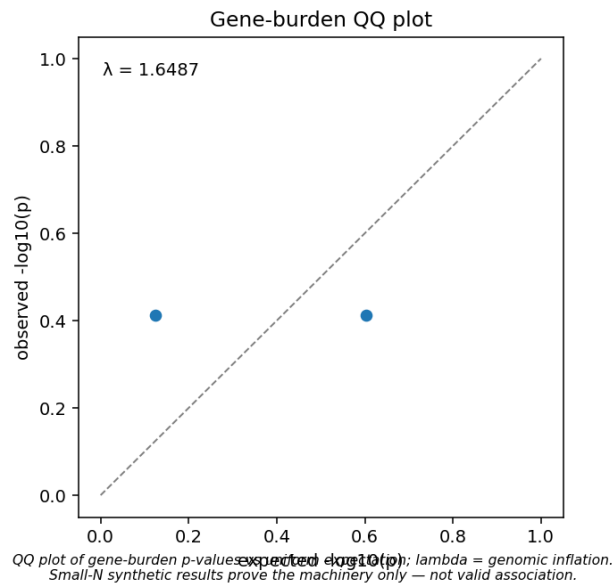


Figure 33: QQ plot of gene-burden p-values vs uniform expectation; lambda = genomic inflation. Small-N synthetic results prove the machinery only — not valid association.

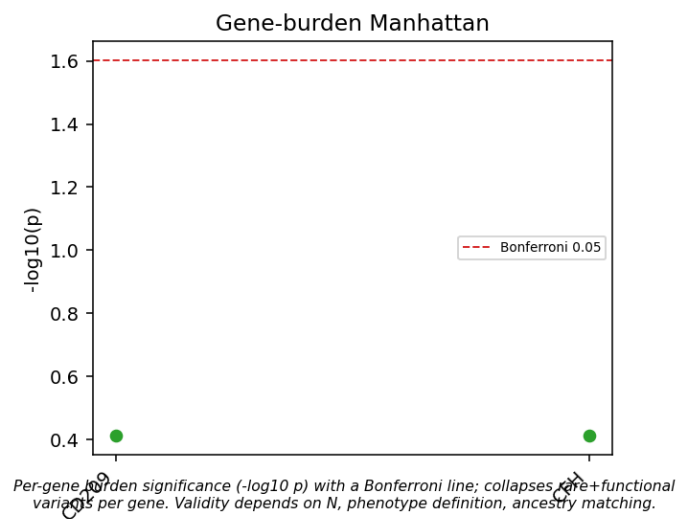


Figure 34: Per-gene burden significance ($-\log_{10} p$) with a Bonferroni line; collapses rare+functional variants per gene. Validity depends on N, phenotype definition, ancestry matching.

11. Outputs explained

Output	Path	Notes
Annotated joint VCF	stage11_annotation/annotatedVEP.GSQ + gnomAD_* + ClinVar_* + PhyloP + paralog flags	
Per-variant TSV	stage11_annotation/variants.flat.tsv	flattened columns below
Filtered joint VCF	stage7_filter/joint.filteredVCF	FILTER = PASS / hard-filter name
CNV calls	stage8_cnv/cnv_calls.tsv	with cnv_callable + confidence
STR genotypes	stage9_str/str_calls.tsv	repeat units, read support, call rate
Paralog VCF	stage10_paralog/paralog.annoINFEEDBACK_GENE, PARALOG_CONF	
Burden results	stage14_burden/burden_results.tsv	engine column stamps the method
Cohort QC	stage12_cohortqc/cohort_qc.json	relatedness, sex, missingness, PCA
GIAB metrics	stage13_giab/plots/giab_metrics/precision/recall/F1, on-target	
Provenance	stage15_report/provenance.json	versions, resources, quarantine, PC gate
Dashboard	stage15_report/cohort_qc_dashboard.html	all data in tables

Per-variant TSV columns (variants.flat.tsv): chrom, pos, ref, alt, rsID, filter, gene, consequence, gnomAD_AF, gnomAD_AF_afr, ClinVar_CLNSIG, PhyloP, paralog_gene, paralog_conf.

Burden results columns (burden_results.tsv): unit, kind, engine, n_carriers, case_carriers, control_carriers, chi2, p (the engine column means a collapse result can never be mistaken for a regenie association when the file is read on its own).

12. Interpreting results

Machinery-proven vs needs-real-cohort. On the synthetic `test` cohort the pipeline *runs* and the *invariants* fire, but the *numbers* are not data quality. Treat as machinery-proven (not real): on-target/enrichment/uniformity, Ti/Tv & het:hom, GIAB precision/recall, all burden association (small N), and relatedness/ancestry structure.

CaptureForge confidence labels. Read CNV calls *through* their callability label: depth-callable genes (e.g. GYPC, HP) are trustworthy; breakpoint-blind genes (CR1, CFH) are `breakpoint_blind_low_confidence` — treat their CNV calls cautiously. Paralog-region variants are split **confident** (uniquely callable, high MQ) vs **flagged** (multimapping ambiguity) — confirm flagged calls orthogonally.

Burden validity. Small cohorts show **genomic inflation** (high λ in the QQ plot) — not signal. The **ancestry-PC gate** drops unstable PCs and runs **unadjusted** (stated in `provenance.json` and the burden summary); an unadjusted run is a deliberate, declared choice, not a silent one. Burden validity depends on N, phenotype definition, and ancestry matching — CallForge does not imply publication-ready association on a small cohort.

GIAB. Precision = fraction of called variants that are true; recall = fraction of truth recovered; both **on-target** (panel-BED-restricted). Genome-wide scoring would distort a panel’s measured sensitivity.

13. Real-run checklist

Before the production cohort:

- ☐ Demultiplexed FASTQs + sample sheet (with **phenotype** for burden).
- ☐ **Genome-wide gnomAD (AFR), ClinVar, dbSNP, PhyloP** — each must pass the scope gate. dbSNP is needed in **two** places (annotation rsIDs + BQSR known-sites).
- ☐ BQSR known-sites via `--known_sites` (dbSNP + Mills + 1000G indels).
- ☐ **1000G GRCh38 somalier sites** via `--somalier_sites`.
- ☐ GIAB control in the sheet + truth v4.2.1 (`--giab_control_id`, `--giab_truth_vcf/bed`).
- ☐ **Confirm ancestry PCs are stable** before using them as burden covariates (the gate enforces this; if unstable it runs unadjusted and says so).
- ☐ **Validate-before-production engines**: DeepVariant+GLnexus, regenie, SKAT-O/STAAR.
- ☐ Re-generate the genome-wide gnomAD panel slice on the HPC (fast network).
- ☐ Revisit GATK hard-filter thresholds for targeted data.

14. Troubleshooting

Symptom	Cause / fix
Scope-gate FAIL on a DB	region-subset DB — supply a genome-wide replacement (message names it)
Annotation values missing	chr-naming mismatch — handled by reconciliation; <code>verify_annotation.py</code> fails loud if not, check the manifest naming
VEP “CacheDir” error in gtf mode	do not pass <code>--offline</code> with <code>--gtf</code> (cache mode only)
mosdepth “exec format error” (arm64)	use mosdepth $\geq 0.3.11$ or <code>CONDA_SUBDIR=osx-64</code>
<code>grep -P</code> errors in fetch script (macOS)	BSD grep lacks <code>-P</code> ; the shipped script uses <code>awk</code> (keep it in sync)
No samples reach calling	all quarantined — read <code>qc_quarantine.txt</code> ; relax thresholds only with justification
Unstable ancestry PCs	the gate drops them and runs unadjusted (recorded); do not force them
Truncated <code>.bgz</code> DB	BGZF-EOF check flags it <code>truncated</code> — re-fetch (resumable)
Hard-filter over-removes on a panel	tune <code>--snp_filter_expr/--indel_filter_expr</code> (defaults are WGS/WES)

15. Extending CallForge

- **New annotation databases:** add them to a vcfanno TOML (`--vcfanno_toml`); the scope gate + landing check apply.
- **New organism:** `--species` + VEP cache + vcfanno TOML; human DBs skip (§6.6).
- **Engine choices:** §8.8 — note the validate-before-production engines.
- **Roadmap (not yet validated):** end-to-end runs of DeepVariant+GLnexus, regenie, and SKAT-O on a real cohort; VQSR for large cohorts; manifest-driven BQSR known-sites.

16. Reproducibility & provenance

Every run writes `stage15_report/provenance.json`: pipeline version + git commit, the reference + invariant statement, the resource manifest (status/build/md5/AFR), QC thresholds + pass/quarantine lists with reasons, the burden status + **ancestry-PC gate decision**, and pointers to the pinned `env/*.yaml` and the pinned VEP/hap.py containers. Pin Nextflow via `manifest.nextflowVersion` and tool versions via the env files; cite the exact reference and database releases recorded in the manifest.

17. Glossary & FAQ

gVCF — per-sample genomic VCF with reference-confidence blocks, the input to joint genotyping. **PoN** — panel-of-normals (CNVkit reference from QC-pass samples). **FREEMIX** — Verify-BamID2 contamination estimate. **FOLD_80** — uniformity penalty (1.0 = uniform). λ (**lambda**) — genomic inflation factor. **Burden test** — gene/gene-set aggregation of rare variants vs phenotype.

FAQ.

- *Do I need CaptureForge to run CallForge?* No — you can supply `--target_bed + --gene_metadata` directly; but the per-gene labels (CNV callability, paralog, burden groups) come from CaptureForge metadata and enrich the analysis.
- *What if I have no phenotype?* The pipeline runs through the annotated callset and skips burden with a message.
- *Why did a sample disappear from the callset?* It was quarantined — see `qc_quarantine.txt`.

18. Citation & the CaptureForge $\leftrightarrow$ CallForge pairing

Cite via `CITATION.cff` in the repository root. CallForge is the **analysis counterpart to CaptureForge**: CaptureForge designs the panel and records per-gene design knowledge; CallForge consumes that handoff and produces the analysed, annotated, QC'd callset and burden layer. Run them as a pair for a panel: design $\rightarrow$ sequence $\rightarrow$ **CallForge**.